

AudioFab: Building A General and Intelligent Audio Factory through Tool Learning

Cheng Zhu
Hunan University
Changsha, Hunan, China
zhucheng@hnu.edu.cn

Kehan Wang
Hunan University
Changsha, Hunan, China
wangkh@hnu.edu.cn

Jing Han*
University of Cambridge
Cambridge, UK
jh2298@cam.ac.uk

Huan Zhao
Hunan University
Changsha, Hunan, China
hzhao@hnu.edu.cn

Qianshuai Xue
Hunan University
Changsha, Hunan, China
xueqianshuai@hnu.edu.cn

Zixing Zhang*
Hunan University
Changsha, Hunan, China
zixingzhang@hnu.edu.cn

Abstract

Currently, artificial intelligence is profoundly transforming the audio domain; however, numerous advanced algorithms and tools remain fragmented, lacking a unified and efficient framework to unlock their full potential. Existing audio agent frameworks often suffer from complex environment configurations and inefficient tool collaboration. To address these limitations, we introduce *AudioFab*, an open-source agent framework aimed at establishing an open and intelligent audio-processing ecosystem. Compared to existing solutions, AudioFab's modular design resolves dependency conflicts, simplifying tool integration and extension. It also optimizes tool learning through intelligent selection and few-shot learning, improving efficiency and accuracy in complex audio tasks. Furthermore, AudioFab provides a user-friendly natural language interface tailored for non-expert users. As a foundational framework, AudioFab's core contribution lies in offering a *stable and extensible platform for future research and development in audio and multimodal AI*. The code is available at <https://github.com/SmileHnu/AudioFab>.

CCS Concepts

• **Software and its engineering** → **Open source model**; • **Information systems** → **Multimedia information systems**.

Keywords

Open source, audio agent, multimedia software, tool learning

ACM Reference Format:

Cheng Zhu, Jing Han, Qianshuai Xue, Kehan Wang, Huan Zhao, and Zixing Zhang. 2025. AudioFab: Building A General and Intelligent Audio Factory through Tool Learning. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, October 27–31, 2025, Dublin, Ireland. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3746027.3756869>

*Jing Han and Zixing Zhang are corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '25, Dublin, Ireland

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2035-2/2025/10
<https://doi.org/10.1145/3746027.3756869>

1 Introduction

In recent years, audio technology has become increasingly important in both daily life and industrial applications. Numerous powerful professional tools have emerged for tasks such as voice interaction, audio restoration, and music production [1–3]. However, most audio tools are either single-function or combine multiple functions, making it hard to create a unified workflow that is both versatile and specialized. Complex, multi-stage tasks (e.g., end-to-end processing from audio analysis to music creation) often require switching tools and models based on different needs, complicating operations and reducing user-friendliness for non-professionals. Moreover, current intelligent solutions lack an efficient collaborative mechanism for task decomposition, tool selection, and execution feedback, hindering the integration of diverse audio tools into real-world workflows and failing to meet practical needs.

In this context, the advent of Large Language Models (LLMs) and the concept of *Tool Learning* [11] offers a new paradigm. By coordinating key stages of tool learning, LLMs can use external tools to complete complex tasks, leading to the creation of LLM-based agents (e.g., METAGPT [4]).

However, compared to rapid advancements in NLP [10] and CV [6], audio agent technology faces significant challenges. First, the *limited functional coverage* of current audio agents restricts their ability to meet the diverse demands of modern audio processing. Second, although many complex tasks can be solved with a few well-chosen tools, existing systems burden LLMs with entire tool libraries, resulting in *excessive contextual information* and increasing the risk of interpretive errors (*aka* tool hallucination), which may escalate into task hallucination and generate misleading outputs. Finally, tool modules in current systems are often tightly coupled and lack dedicated management mechanisms, making it *difficult to add or update tools* without triggering dependency conflicts or version incompatibilities, thus hindering sustainable development.

To address these challenges, we present *AudioFab*, a *general and intelligent audio factory designed for audio-centred multimodal applications*, which introduces improvements in *functional breadth, tool management, and Tool Learning strategies*. The main contributions of this work are as follows:

- **A specialized audio tool library with modular management:** we collect a comprehensive set of audio-related tools for building

Table 1: AudioFab and Comparable Software. ✓ = supported, ✗ = unsupported, “None” = unknown, *MM*: Module Management, *Edt.*: Editing, *Und.*: Understanding, *Gen.*: Generation, *Bat.*: Audio Batch Input, *MCP*: Model Context Protocol.

	Platforms	MM	Edt.	Und.	Gen.	Bat.
AudioGPT [7]	LangChain	✗	✗	✓	✓	✓
WavCraft [8]	None	✗	✓	✗	✓	✗
WavJourney [9]	None	✗	✓	✗	✓	✗
AudioFab (ours)	MCPs	✓	✓	✓	✓	✓

a specialized audio tool library and introduce a modular management mechanism, supporting extensibility and mitigating dependency conflicts.

- **An intelligent tool learning process:** we develop a tool learning and selection strategy that dynamically adjusts context length based on user input and leverages few-shot learning to provide usage examples for selected tools, enabling LLMs to identify and invoke tools more effectively.
- **A unified and user-friendly audio agent framework:** we build an audio-specific agent framework using the Model Context Protocol (MCP), providing a unified interface and simplifying access for non-expert users via natural language interaction.

AudioFab is released as an *open-source* toolkit under the *CC BY-NC 4.0* license and actively maintained on GitHub. Built in Python, it integrates a range of dependencies, including PyTorch, audio processing libraries, and web frameworks. It supports *easy installation* via a one-command Conda setup. The toolkit provides *clear instructions* for adding new APIs or custom audio tools, and comes with *comprehensive examples and documentation* in both English and Chinese, to support a broad user community.

2 Related Work

Several initial efforts in this area have been made by prior works. For instance, AudioGPT was first introduced in [7]. It integrates multiple specialized neural network models and complements LLM for task classification based on user queries, thus supporting understanding and generating audio modality. Likewise, frameworks such as WavCraft [8] and WavJourney [9] register multiple model-based tools as functions and leverage the programming capabilities of LLM to automatically generate Python scripts for audio processing.

While these systems demonstrate the feasibility of combining LLMs with audio tools, they face three key limitations. *First*, they do not span the full lifecycle or cover the broad spectrum of audio analysis tasks. *Second*, extensibility is limited—for example, WavJourney depends on structured audio scripts and script compilers, making it difficult to expand functionality or incorporate new tools. *Third*, most frameworks provide only a single pre-registered model per task, restricting adaptability to emerging tasks or alternative algorithms. Overall, the absence of a modular architecture and platform support limits their flexibility and scalability.

In contrast, our work, **AudioFab**, introduces notable advancements in implementation strategy, functional scope, and module management. It supports the full workflow, from audio processing

and analysis to generation and conversion, offering a comprehensive suite of capabilities. Additionally, we propose a dedicated module management mechanism to enhance extensibility and control. A detailed comparison is provided in Table 1.

3 AudioFab: Design & Features

Figure 1 presents the architecture of AudioFab, which comprises three core module components: the yellow block (MCP Client and LLM Planner), the orange block (MCP Server and Tool Selection), and the pink block (Process Selection). These components connect end-users (green blocks) with audio tools (blue blocks). Powered by a user-friendly, LLM-driven interface, the system enables collaborative and interoperable workflows, effectively supporting complex audio processing tasks. Next, we provide a detailed discussion of its key features, including the module management, tool learning process, and functional scope within the audio domain.

3.1 Module Management

AudioFab adopts the recent *MCP protocol* [5] to support its highly modular architecture by standardizing how audio tools interact with LLMs. In particular, the *MCP Client* serves as a unified external interface, facilitating communication between users, the LLM, and the MCP Server. Meanwhile, the *MCP Server* acts as the internal management hub, coordinating tool execution across various environments using standardized protocols. Each tool operates in an isolated environment (e. g., Conda, Docker). This prevents dependency conflicts and improves system stability and maintainability.

3.2 Tool Learning

A key innovation that sets AudioFab apart from comparable systems is its tool learning process, embedded within the full workflow. This process unfolds in four sequential stages. Below, we describe each stage and reference the corresponding steps illustrated in Figure 1.

Task Planning: at this stage, the MCP Client serves as the user interface, receiving user commands (*step 1*) and forwarding them to the LLM (*step 2*). The LLM, as the system’s core planner, interprets the request, decomposes it into structured subtasks, and returns them to the client (*step 3*). Unlike traditional linear ReACT processes, AudioFab employs specialized prompts and a feedback mechanism tailored for audio tasks, enabling dynamic evaluation and adjustment for improved robustness and accuracy.

Tool Selection: next, the MCP Client forwards the request to the MCP Server (*step 4*), which acts as an adapter by translating tool calls into commands the tools can understand. These commands are then passed to the Tool Selection Module (*step 5*). This module includes three parts: tool enumeration, retrieval matching, and parameter querying. It uses a JSON file containing tool names, descriptions, and examples. During initialization, only tool instructions are loaded into the LLM to save tokens. When a user issues a tool-related request (*step 6*), semantic matching identifies the most relevant toolchains, returning corresponding usage instructions and examples (*step 7*). With few-shot learning, this helps ensure accurate selection and reduces errors or hallucinations.

Tool Invocation: this stage includes two phases, namely Tool Call and Tool Execution. The LLM evaluates the toolchain provided by the selection module and submits a structured request (*step*

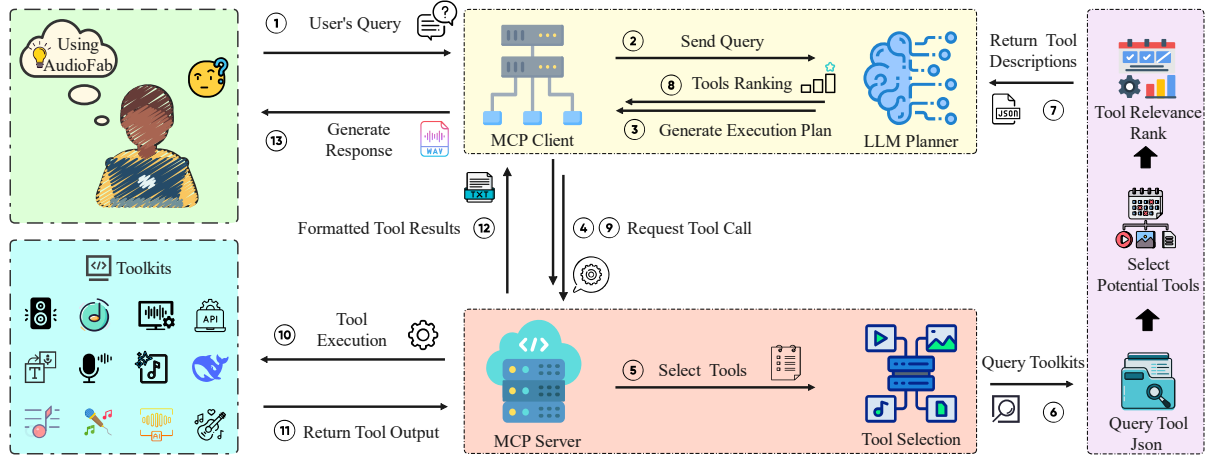


Figure 1: Overview of the AudioFab pipeline. Colored blocks represent architectural components, while arrows illustrate the workflow. With AudioFab, audio tools and end-users are connected through a comprehensive, user-friendly interface, enabling closer and more effective collaboration than ever before.

Table 2: AudioFab supports 36 key audio techniques. Techniques are grouped into *editing*, *understanding*, and *generation* (further split into *Audio2Audio*, *X2Audio*, and *Audio2X*). Capabilities are shown across *speech*, *general sound*, and *music* domains, respectively.

Modal	Editing	Understanding	Generation		
			Audio2Audio	X2Audio	Audio2X
Speech	Speech Editing, Speech Enhancement, Voice Activity Detection, Speech Super Resolution	Speech Translation, Speech Emotion Recognition, Speaker Diarization, Speaking Style Recognition	Speech2Song, Spoken Dialogue, Voice Conversion	Text2Speech, Target Speaker Extraction	ASR, AAC, Speech2Talking-head
Sound	Digital Signal Processing, Audio Reconstruction, Audio Separation	Sound Event Detection, Audio Quality Analysis, Acoustic Scene Classification	Sound Style Transfer	Text2Audio, Video2Audio	Audio2Video, Audio2Image
Music	Music Separation, Music Mix Track, Music Format Conversion	Music Emotion Recognition, Music Style Description	Music2Song	Text2Music, Lyrics2Song	Lyrics Recognition

ASR: Automatic Speech Recognition, AAC: Automated Audio Captioning

8). The MCP Server handles the request (step 9) through unified invocation and execution (step 10), then returns the tool outputs back to the server (step 11).

Response Generation: finally, once all subtasks are completed, formatted tool results are sent to the LLM via the MCP Client (step 12). The LLM aggregates and synthesizes these outputs into a coherent response and returns it to the user through the MCP Client (step 13), completing the end-to-end workflow.

3.3 Comprehensive and Growing Capabilities

Building on the modular architecture and tool learning mechanisms described above, AudioFab enables flexible integration, expansion, and management of diverse audio toolkits. The system currently supports 36 audio tasks, spanning from basic operations like format conversion and noise reduction to advanced applications such as voice conversion and cross-modal generation. These capabilities, summarized in Table 2, reflect the system’s breadth and depth.

Furthermore, AudioFab is designed to continuously evolve, allowing seamless addition of new tools as needs arise. *We believe this is just the beginning of AudioFab’s journey, with ample potential for future expansion and continued impact in the audio AI and multimodal research community.*

4 Evaluation with AudioFab

Due to the absence of standardized benchmarks for evaluating AudioFab’s broad capabilities, we assess its performance through three illustrative examples, each representing a key use case. These diverse scenarios are carefully chosen to: (i) test across different modalities, (ii) validate its support for chained workflows involving multiple interconnected tools, and (iii) show its ability to edit, understand, and generate audio. The versatility of AudioFab is demonstrated through these examples, as illustrated in Figure 2.

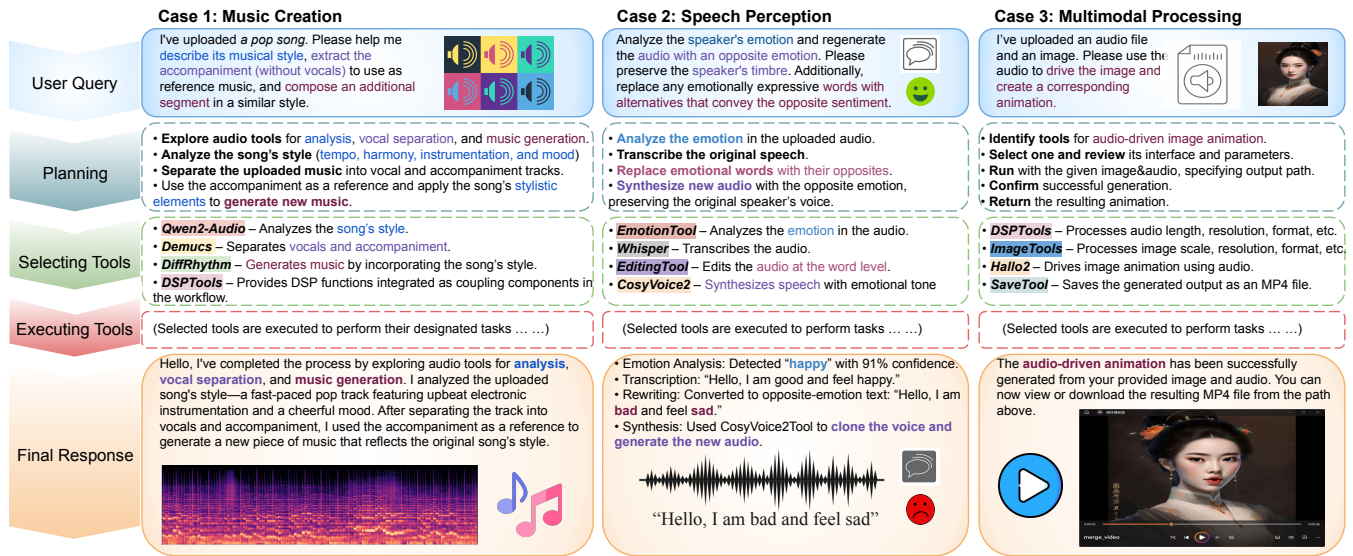


Figure 2: Three illustrative usage scenarios of the AudioFab toolkit: (1) music creation, (2) speech editing, and (3) multimodal interaction. Each scenario demonstrates a prototypical workflow—from user query and plan generation to tool selection and final output, showcasing the toolkit's versatility, structured execution, and practical applicability.

Music Creation: this case showcases AudioFab's music creation capabilities. Given a pop song, the system analyzes its style, separates vocals from accompaniment, and generates new segments in a similar style using the accompaniment as reference.

Speech Perception: this case indicates AudioFab's speech perception capabilities. Given a speech clip, the system recognizes emotion, transcribes the content, replaces emotional terms with their opposites, and regenerates speech with reversed emotion while preserving the original timbre.

Multimodal Processing: this case highlights AudioFab's multimodal processing capabilities. Given an image and audio input, the system generates audio-driven animation by transforming the image based on audio dynamics and synthesizing a coherent video.

In summary, these three use cases confirm AudioFab's effectiveness as a specialized toolset and tool learning solution, showcasing its potential as a unified platform for intelligent audio processing.

5 Conclusion and Future Work

In this paper, we present *AudioFab*, an open-source unified audio agent framework covering three core modalities: speech, sound, and music. It integrates 36 key functionalities through an intuitive natural language interface. The tool serves two key audiences: for *non-experts*, it eliminates the need for specialized domain knowledge to handle complex audio tasks, while for *researchers and developers*, it reduces overhead in tool deployment, facilitating deeper exploration and more rapid prototyping. Ultimately, this solution lowers the field's entry barrier for practitioners across all levels, making audio technologies accessible to a broader audience.

In the future, we aim to establish a comprehensive benchmark and leaderboard to assess the framework's reliability and quality, maintained and updated on GitHub.

Acknowledgments

This work was supported by the Guangdong Basic and Applied Basic Research Foundation under Grant No. 2024A1515010112, the Changsha Science and Technology Bureau Foundation under Grant No. kq2402082, and the National Natural Science Foundation of China under Grant No. 62076092.

References

- [1] Mohammed M Alghamdi, He Wang, Andrew J Bulpitt, and David C Hogg. 2022. Talking head from speech audio using a pre-trained image generator. In *Proc. ACM International Conference on Multimedia (MM)*. Lisboa, Portugal, 5228–5236.
- [2] Yunkee Chae, Junghyun Koo, Sungho Lee, and Kyogu Lee. 2023. Exploiting time-frequency Conformers for music audio enhancement. In *Proc. ACM International Conference on Multimedia (MM)*. Ottawa, Canada, 2362–2370.
- [3] Deepanway Ghosal, Navonil Majumder, Ambuj Mehrish, and Soujanya Poria. 2023. Text-to-audio generation using instruction guided latent diffusion model. In *Proc. ACM International Conference on Multimedia (MM)*. Ottawa, Canada, 3590–3598.
- [4] Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *Proc. International Conference on Learning Representations (ICLR)*. Vienna, Austria, 1–14.
- [5] Xinyi Hou, Yanjie Zhao, Shenao Wang, et al. 2025. Model context protocol (MCP): Landscape, Security threats, and Future Research Directions. *arXiv preprint arXiv:2503.23278* (2025).
- [6] Zhangli Hu, Ye Chen, Zhongyin Zhao, et al. 2024. Towards Artist-Like Painting Agents with Multi-Granularity Semantic Alignment. In *Proc. ACM International Conference on Multimedia (MM)*. Melbourne, Australia, 10191–10199.
- [7] Rongjie Huang, Mingze Li, Dongchao Yang, et al. 2024. AudioGPT: Understanding and Generating Speech, Music, Sound, and Talking Head. In *Proc. AAAI Conference on Artificial Intelligence*. Vancouver, Canada, 23802–23804.
- [8] Jinhua Liang, Huan Zhang, Haohe Liu, et al. 2024. Wavcraft: Audio editing and generation with natural language prompts. In *Proc. International Conference on Learning Representations (ICLR)*. Vienna, Austria, 75–92.
- [9] Xubo Liu, Zhongkai Zhu, Haohe Liu, et al. 2025. WayJourney: Compositional Audio Creation with Large Language Models. *IEEE Transactions on Audio, Speech and Language Processing* 1 (2025), 1–15.
- [10] Bo Ni and Markus J Buehler. 2024. MechAgents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. *Extreme Mechanics Letters* 67 (2024), 102131.
- [11] Changle Qu, Sunhao Dai, Xiaochi Wei, et al. 2025. Tool learning with large language models: A survey. *Frontiers of Computer Science* 19, 8 (2025), 198343.